

Algoritmo RemoverKLEnc

```
Entradas: PtLista (TipoPtNode)
          K (inteiro)
Saídas: PtLista (TipoPtNode)
        Sucesso (lógico)
Variáveis auxiliares: PtAnt, PtK (TipoPtNode)
início
se K < 1
então Sucesso ← falso
senão início
    PtK ← PtLista
    PtAnt ← nulo
    enquanto (PtK ≠ nulo) e (K > 1)
    faça início
        K ← K-1
        PtAnt ← PtK
        PtK ← PtK↑.Elo
    fim
se PtK = nulo
então Sucesso ← falso
senão início
    se PtK = PtLista
    então PtLista ← PtLista↑.Elo
    senão PtAnt↑.Elo ← PtK↑.Elo
    liberar(PtK)
    Sucesso ← verdadeiro
fim      fim      fim
```

```
void remover(LISTA **PtLista, int *count){
    LISTA *PtAnt, *PtK;
    int K;

    printf("\nDigite qual posição deseja remover: ");
    scanf("%d", &K);

    if(K < 1 || *PtLista == NULL){
        printf("\nNão é possível remover!\n");
        return;
    }

    PtK = *PtLista;
    PtAnt = NULL;

    while(PtK != NULL && K > 1){
        K--;
        PtAnt = PtK;
        PtK = PtK->elo;
    }

    if(PtK == NULL){
        printf("\nNão é possível remover!\n");
    }
    else{
        if(PtAnt == NULL) // remove o primeiro nó
            *PtLista = PtK->elo;
        else // remove do meio ou final
            PtAnt->elo = PtK->elo;

        free(PtK);
        (*count)--;
        printf("\nRemovido com sucesso!\n");
    }
}
```